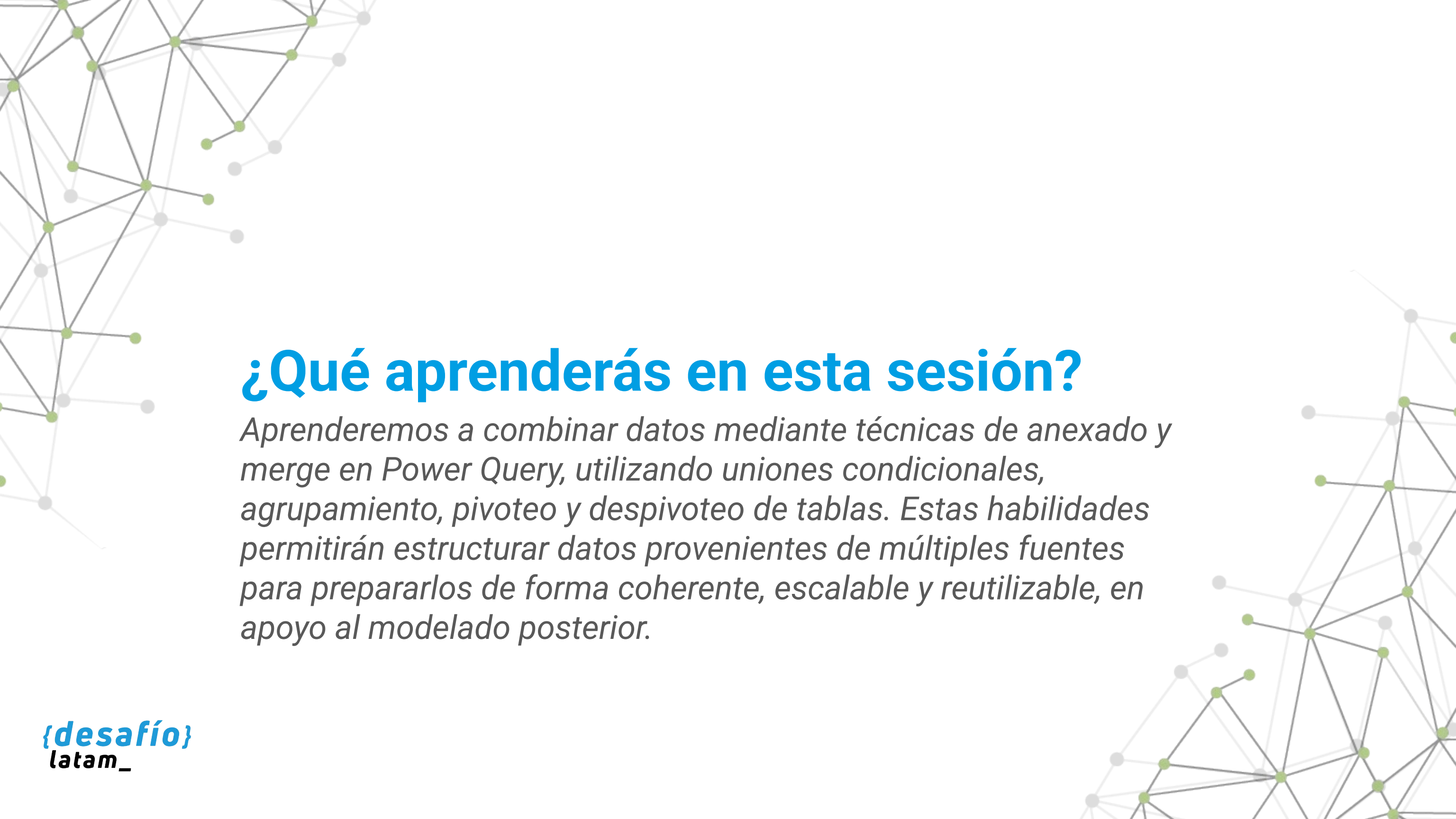




Preparación, Modelado y DAX

Clase 3 - Combinación de datos con Power Query



¿Qué aprenderás en esta sesión?

Aprenderemos a combinar datos mediante técnicas de anexado y merge en Power Query, utilizando uniones condicionales, agrupamiento, pivoteo y despivoteo de tablas. Estas habilidades permitirán estructurar datos provenientes de múltiples fuentes para prepararlos de forma coherente, escalable y reutilizable, en apoyo al modelado posterior.

Desafío inicial

Trabajas como analista de datos para una red nacional de servicios públicos. Cada sucursal regional envía mensualmente archivos separados con información de atenciones, y además existen otras fuentes que contienen catálogos de servicios, tablas de referencia de comunas y encuestas de satisfacción. Tu tarea es construir una tabla consolidada que integre todos estos orígenes para poder crear un dashboard único de gestión.

Sin embargo, los archivos tienen distintas estructuras: algunos con columnas en distinto orden, otros con nombres diferentes para el mismo campo, y algunos datos están en formato "largo" y otros en formato "ancho".

¿Cómo unificas y transformas estas estructuras sin perder trazabilidad ni calidad?

Esta sesión te dará las herramientas para lograrlo.



/* Anexado de datos (Append Queries)
***/**

Anexado de datos (Append Queries)

¿Por qué anexar datos en procesos de integración?

En entornos organizacionales reales, los datos se generan de manera descentralizada: por sucursal, periodo, canal de atención o sistema fuente. Esta fragmentación obliga a implementar flujos de integración que permitan consolidar múltiples archivos o tablas con estructuras equivalentes.

El anexado de datos (append) permite apilar registros horizontalmente, creando una única tabla estructurada para análisis agregados. Esta operación se utiliza ampliamente en procesos de ETL (extracción, transformación y carga), sobre todo en la fase de "preparación de datos", previa al modelado analítico en Power BI.

✕

✓

f_x

Table.Combine({#"Changed column type", #"Store Sales", #"Wholesale Sales"})

	ABC Channel Name	📅 Date	ABC CustomerID	123 Units	ABC Referrer
1	Online	6/2/2024	US-187982	20	null
2	Online	6/2/2024	US-235789	30	null
3	Online	6/5/2024	US-187982	15	null
4	Online	6/5/2024	US-235789	40	null
5	Store A	6/4/2024	e-78956	200	www.powerbi.com
6	Store B	6/4/2024	e-78899	300	www.powerquery.com
7	Store A	6/7/2024	e-75214	100	www.xbox.com
8	Store B	6/7/2024	e-23658	70	www.microsoft.com
9	Wholesale	6/1/2024	V-8884	5000	null
10	Wholesale	6/1/2024	V-1234	4000	null
11	Wholesale	6/1/2024	V-2943	6000	null
12	Wholesale	6/1/2024	V-8893	4000	null

Tipos de anexo en Power Query y su uso estratégico

Tipo de Append	Descripción	Escenario de uso
Anexo como nueva consulta	Crea una nueva consulta resultado de la unión de varias tablas.	Cuando se desea preservar la integridad de las fuentes originales.
Anexo dentro de una consulta existente	Inserta los registros de otras tablas directamente en una consulta activa.	Útil en flujos simples o cuando ya se está construyendo una consulta base.

Tabla 1. Tipos de anexo en Power Query y su uso estratégico

Fuente: Desafío Latam



Nota técnica: Power Query no requiere que las tablas tengan la misma cantidad ni el mismo orden de columnas. Las columnas inexistentes se completarán con valores null.

Requisitos técnicos para anexo sin errores

Para evitar inconsistencias semánticas o errores durante la carga, se deben cumplir las siguientes condiciones:

Homogeneidad de nombres de columnas

Todos los datasets deben tener los mismos encabezados.

Uniformidad de tipo de dato

Una misma columna no debe contener texto en una fuente y número en otra.

Orden semántico

Aunque Power Query no exige el mismo orden de columnas, conviene mantenerlo por trazabilidad.

Identificación del origen

Se recomienda agregar una columna adicional como Fuente o Periodo para mantener rastreabilidad.

 Métrica clave: Al realizar un append exitoso, la métrica de validación es que Número de columnas x Número de filas sea coherente con la suma de los registros esperados y sin creación de columnas residuales con valores nulos.

Buenas prácticas y errores comunes

Buenas prácticas:

- Antes de anexar, transformar todos los nombres de columnas a un mismo formato: sin espacios, sin tildes, en minúsculas (Text.Lower, Text.Trim).
- Documentar el paso con nombres autoexplicativos en Power Query (por ejemplo: Append_Solicitudes_Mensuales).
- Cargar cada tabla fuente como una consulta independiente y aplicar limpieza antes del append.

Errores comunes:

- Anexar tablas con nombres similares pero no idénticos en columnas (ej. "Tipo solicitud" vs "TipoSolicitud"), generando columnas duplicadas y con valores vacíos.
- Omitir la estandarización del tipo de dato, lo que puede provocar errores de modelado posteriores.
- No registrar el origen de cada fila, lo que impide realizar trazabilidad o análisis por periodo.

Ejemplo aplicado con código Power Query (lenguaje M)

Supongamos que se cuenta con tres archivos mensuales: enero.xlsx, febrero.xlsx, marzo.xlsx. Cada archivo contiene la tabla Solicitudes con columnas estandarizadas: Fecha, Comuna, TipoSolicitud, Canal.

```
let
    // Cargar archivos individualmente
    Enero = Excel.Workbook(
        File.Contents("C:\\Datos\\enero.xlsx"),
        null,
        true
    ){[Item="Solicitudes", Kind="Sheet"]}[Data],

    Febrero = Excel.Workbook(
        File.Contents("C:\\Datos\\febrero.xlsx"),
        null,
        true
    ){[Item="Solicitudes", Kind="Sheet"]}[Data],

    Marzo = Excel.Workbook(
        File.Contents("C:\\Datos\\marzo.xlsx"),
        null,
        true
    ){[Item="Solicitudes", Kind="Sheet"]}[Data],

    // Agregar columna de origen
    EneroConMes = Table.AddColumn(Enero, "Mes", each "Enero"),
    FebreroConMes = Table.AddColumn(Febrero, "Mes", each "Febrero"),
    MarzoConMes = Table.AddColumn(Marzo, "Mes", each "Marzo"),

    // Estandarizar nombres de columnas (opcional)
    EneroFinal = Table.TransformColumnNames(EneroConMes, Text.Trim),
    FebreroFinal = Table.TransformColumnNames(FebreroConMes,
        Text.Trim),
    MarzoFinal = Table.TransformColumnNames(MarzoConMes, Text.Trim),

    // Realizar append
    SolicitudesUnificadas = Table.Combine({
        EneroFinal,
        FebreroFinal,
        MarzoFinal
    })
in
    SolicitudesUnificadas
```

Resultado esperado: Una tabla única con columnas: Fecha, Comuna, TipoSolicitud, Canal, Mes, con la suma de filas de los tres meses.

Validación post-anexado

Una vez realizado el append, se deben realizar verificaciones mínimas:

- ¿Se han creado columnas adicionales no deseadas?
- ¿Existen celdas null en campos clave que antes no existían?
- ¿La columna agregada de origen (Mes) está correctamente completada?

Puedes usar el perfil de columna en Power Query para revisar distribuciones, valores únicos y nulos.

El anexado es una de las herramientas fundamentales en la fase de preparación de datos cuando se trabaja con múltiples fuentes con estructuras equivalentes. Permite escalar flujos de trabajo sin duplicación de esfuerzo y mantener consistencia en el modelado.

Una vez que tenemos una tabla consolidada por registros (filas), el siguiente paso lógico es enriquecer la información con atributos provenientes de otras tablas. Para eso, necesitaremos trabajar con operaciones de merge, que permiten relacionar tablas a través de claves. Veamos cómo aplicarlas de forma efectiva en el siguiente bloque.

**/* Operaciones de Merge (Uniones entre
tablas) */**

Operaciones de Merge (Uniones entre tablas)

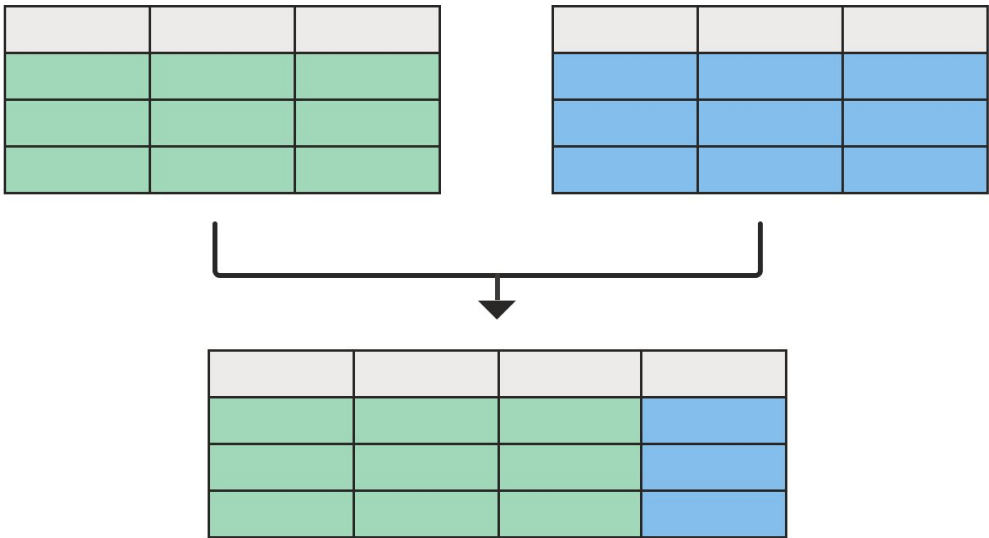
¿Por qué unir tablas en entornos reales?

En los entornos organizacionales, los datos no existen en una única tabla.

Cada conjunto refleja una parte del proceso: ventas, clientes, ubicaciones, servicios, encuestas, etc. Para tener una visión integrada, es fundamental relacionar estas tablas entre sí, estableciendo vínculos semánticos a través de claves comunes.

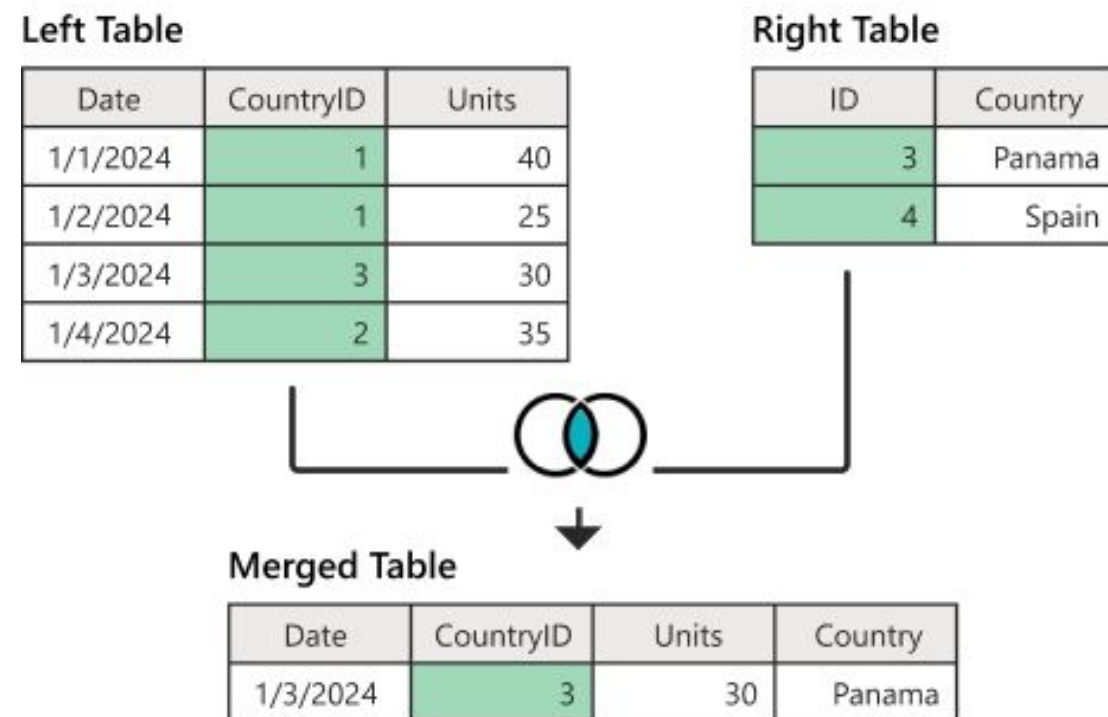
Esta operación se denomina "merge" o unión, y es una práctica estructural en Business Intelligence. Nos permite enriquecer una tabla con atributos adicionales, facilitar validaciones cruzadas, y preparar la base para el modelado multidimensional en Power BI.

Contexto productivo: Un analista de un municipio necesita vincular la tabla de "Solicitudes ciudadanas" con una tabla oficial de "Comunas válidas" para validar errores tipográficos o registros no estandarizados. Sin esta operación, el dashboard de cobertura comunal entregará resultados erróneos o incompletos.



¿Qué es una operación de merge en Power Query?

Una operación de merge es un proceso de combinación de filas entre dos tablas, donde se utiliza uno o más campos en común como criterio de emparejamiento. El resultado es una nueva tabla que contiene todas las columnas de la tabla original (llamada "izquierda") y los campos coincidentes desde la segunda tabla (llamada "derecha").



- Nota técnica: A diferencia de una relación en el modelo de Power BI (Power Pivot), el merge en Power Query se ejecuta como una transformación estática: los valores quedan físicamente integrados en la tabla resultante.

Tipos de uniones disponibles y sus implicancias analíticas

Tipo de Join (JoinKind)	Descripción	Caso de uso frecuente
Left Outer	Mantiene todos los registros de la tabla izquierda, agrega solo coincidencias de la derecha	Validación de claves o enriquecimiento de dimensiones
Right Outer	Lo opuesto al anterior	Poco común, pero útil en exploraciones reversas
Inner Join	Solo registros con coincidencias en ambas tablas	Consolidación de hechos cruzados
Full Outer	Todos los registros de ambas tablas	Diagnóstico de cobertura
Left Anti	Registros de la izquierda sin coincidencia	Identificación de errores o valores inválidos
Right Anti	Registros de la derecha sin coincidencia	Validación de catálogo de referencia

Tabla 2. Tipos de uniones disponibles y sus implicancias analíticas

Fuente: Desafío Latam

 Métrica clave para validación: El conteo de filas post-merge debe ser igual o menor al de la tabla original. Si se duplica, probablemente la clave usada no era única, generando un "many-to-many merge", lo cual debe evitarse.

Requisitos técnicos y semánticos antes de aplicar un merge

Normalización previa de claves

- Aplicar Text.Trim para quitar espacios.
- Usar Text.Lower o Text.Upper para estandarizar mayúsculas/minúsculas.
- Eliminar tildes o caracteres especiales si las fuentes son inconsistentes.

Validación de unicidad en la tabla secundaria

- Verificar que el campo de unión no contenga duplicados si esperamos una relación 1:1.

Homogeneidad de tipos de datos

- Asegurar que los campos de unión tengan el mismo tipo (por ejemplo: ambos text o ambos whole number).

Ejemplo aplicado: Validación de comunas en solicitudes

Tienes:

- Una tabla Solicitudes con una columna Comuna que contiene registros ingresados manualmente.
- Una tabla Comunas_Validas con la columna Nombre que representa comunas oficiales.

Quieres validar que las comunas ingresadas estén en el listado oficial.

```
let
  MergeSolicitudes = Table.NestedJoin(
    Solicitudes,
    {"Comuna"},
    Comunas_Validas,
    {"Nombre"},
    "DatosComuna",
    JoinKind.LeftOuter
  ),
  Expandido = Table.ExpandTableColumn(
    MergeSolicitudes,
    "DatosComuna",
    {"Nombre"},
    {"ComunaValidada"}
  )
in
  Expandido
```

En este ejemplo, las filas donde ComunaValidada es null representan comunas mal escritas o inexistentes. Esto habilita un filtro de control de calidad.

Buenas prácticas



Nombrar claramente el paso de merge

(Merge_ComunasValidadas), indicando propósito y tipo de unión.



Agregar una columna de control post-merge para indicar si hubo coincidencia (if [ComunaValidada] = null then "Inválida" else "Válida").



Evitar hacer merge sobre campos no normalizados: hacerlo incrementa el riesgo de duplicación o pérdida de registros.



Aplicar anti-joins periódicamente para monitorear claves no coincidentes.

Errores comunes y cómo evitarlos

Error	Consecuencia	Solución
Claves con espacios invisibles	No se detectan coincidencias	Aplicar Text.Trim()
Merge con campos duplicados en la tabla derecha	Multiplicación de registros (cartesian join)	Usar campos únicos o consolidar antes
Tipos de datos diferentes	Falla en la unión o coincidencias falsas	Unificar tipo de dato
Merge sin expandir los resultados	Información queda anidada y no visible	Usar ExpandTableColumn

Tabla 3. Errores comunes y cómo evitarlos

Fuente: Desafío Latam

Las uniones entre tablas son el fundamento del análisis multidimensional y la limpieza avanzada de datos. Dominar los tipos de join y los criterios de unión te permitirá construir flujos de datos robustos, escalables y reutilizables.

/* Agrupamiento (Group By) */

Agrupamiento (Group By)

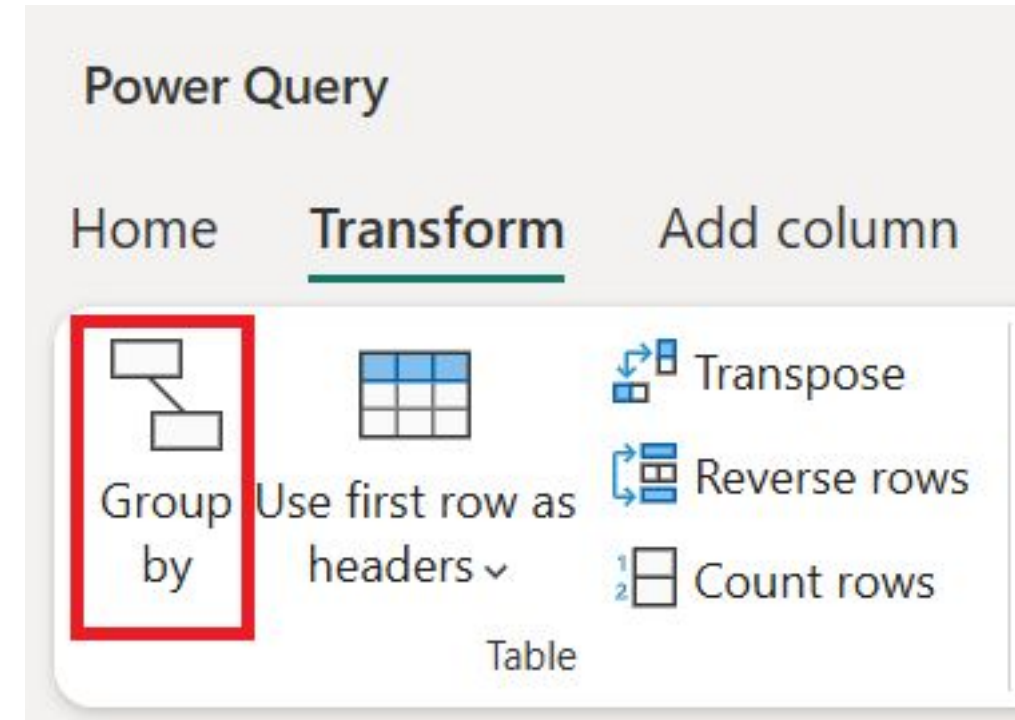
¿Por qué agrupar datos en análisis de negocio?

Una vez que los datos han sido integrados (mediante append o merge), el siguiente paso lógico en la preparación es resumir la información para facilitar el análisis. Esta operación es clave para construir indicadores de desempeño, detectar patrones y reducir la granularidad del dataset.

El agrupamiento permite agregar registros que comparten un valor categórico en común (por ejemplo, comuna, tipo de servicio, canal de atención), aplicando funciones como suma, promedio, conteo o valores extremos.

Contexto productivo: Un encargado de operaciones desea visualizar cuántas solicitudes ciudadanas se realizaron por canal en los últimos tres meses, con el fin de decidir en qué canales reforzar personal o automatizar atención.

Para ello, debe agrupar los datos por "Canal de ingreso" y aplicar una función de conteo.



¿Qué sucede técnicamente al agrupar en Power Query?

La operación de Group By en Power Query genera una nueva tabla, donde cada fila representa un grupo único según las columnas seleccionadas. A cada grupo se le puede aplicar una o varias funciones de agregación, como:

- `Table.RowCount(_)`: contar registros
- `List.Sum([Columna])`: suma de valores
- `List.Average([Columna])`: promedio
- `List.Min/Max([Columna])`: extremos

⚠ Esta operación es destructiva: los datos originales se reemplazan por el resultado agrupado. Por ello, se recomienda duplicar la consulta si se necesita conservar el detalle original.

Subtipos de agrupamiento: simple vs. anidado

Agrupamiento simple

Basado en una o más columnas categóricas con una única operación por grupo.

```
Table.Group(  
    Solicitudes,  
    {"Comuna"},  
    {  
        {"Total", each Table.RowCount(_),  
Int64.Type}  
    }  
)
```

Agrupamiento anidado

Retorna una tabla como columna dentro del grupo, permitiendo operaciones

posteriores como filtrado o nuevas agregaciones dentro del grupo.

```
Table.Group(  
    Solicitudes,  
    {"TipoSolicitud"},  
    {  
        {  
            "Subtabla",  
            each _,  
            type table [Fecha=nullable date, Canal=text, Comuna=text]  
        }  
    }  
)
```

Tipos de funciones agregadas y sus aplicaciones

Función agregada	Propósito en BI	Ejemplo práctico
Count	Medir volumen de eventos	Total de solicitudes por canal
Sum	Acumular métricas	Suma de montos pagados por comuna
Average	Obtener indicadores comparativos	Tiempo promedio de atención
Min/Max	Detectar valores extremos	Fecha más reciente de ingreso por servicio

Tabla 4. Tipos de funciones agregadas y sus aplicaciones

Fuente: Desafío Latam

Buenas prácticas antes y después del agrupamiento

Antes de agrupar:

- Validar que los campos a agrupar estén limpios y sin valores nulos.
- Unificar tipos de datos: fechas, textos, números.
- Verificar que los valores categóricos tengan consistencia semántica (ej. "Call Center" vs. "callcenter").

Después de agrupar:

- Revisar que los resultados sean coherentes con la lógica del negocio.
- Renombrar los campos agregados para que sean comprensibles (evitar nombres como "Column1" o "Count").
- Validar que no se pierdan atributos necesarios para el análisis posterior.

Errores comunes en agrupamiento y cómo evitarlos

Error	Consecuencia	Prevención
Agrupar por campos sucios (espacios, inconsistencias)	División incorrecta en grupos	Aplicar Text.Trim, Text.Lower
No convertir campos numéricos al tipo adecuado	Fallo en agregación (suma, promedio)	Asegurar tipo number o decimal antes de operar
No verificar la semántica del resultado	Generación de KPIs erróneos	Revisar casos de prueba y validar con usuarios clave

Tabla 5. Errores comunes en agrupamiento y cómo evitarlos

Fuente: Desafío Latam

Ejemplo aplicado en Power Query (M)

Agrupar solicitudes por canal y calcular el número total de registros por tipo de solicitud:

```
let
    Agrupado = Table.Group(
        Solicitudes,
        {"Canal", "TipoSolicitud"},
        {
            {"TotalSolicitudes", each Table.RowCount(_), Int64.Type}
        }
    )
in
    Agrupado
```

Este ejemplo genera una tabla donde cada combinación de Canal y TipoSolicitud tiene su conteo de registros asociados. Puede usarse directamente en visualizaciones tipo matriz o gráfico de barras agrupado.

El agrupamiento es una técnica crítica para transformar datos transaccionales en información agregada útil para la toma de decisiones. Permite construir KPIs sólidos, optimizar visualizaciones y reducir el tamaño de los datasets sin pérdida de valor informativo.

Ahora que hemos aprendido a resumir los datos de forma flexible y eficiente, es momento de abordar un aspecto igualmente fundamental: la transformación de la estructura de las tablas, es decir, cómo reorganizar columnas y filas mediante operaciones de pivoteo y despivoteo, para adaptar nuestros datos a distintos formatos de análisis y visualización.

/* Pivoteo y Despivoteo de tablas */

Pivoteo y Despivoteo de tablas (Transformación de estructura tabular)

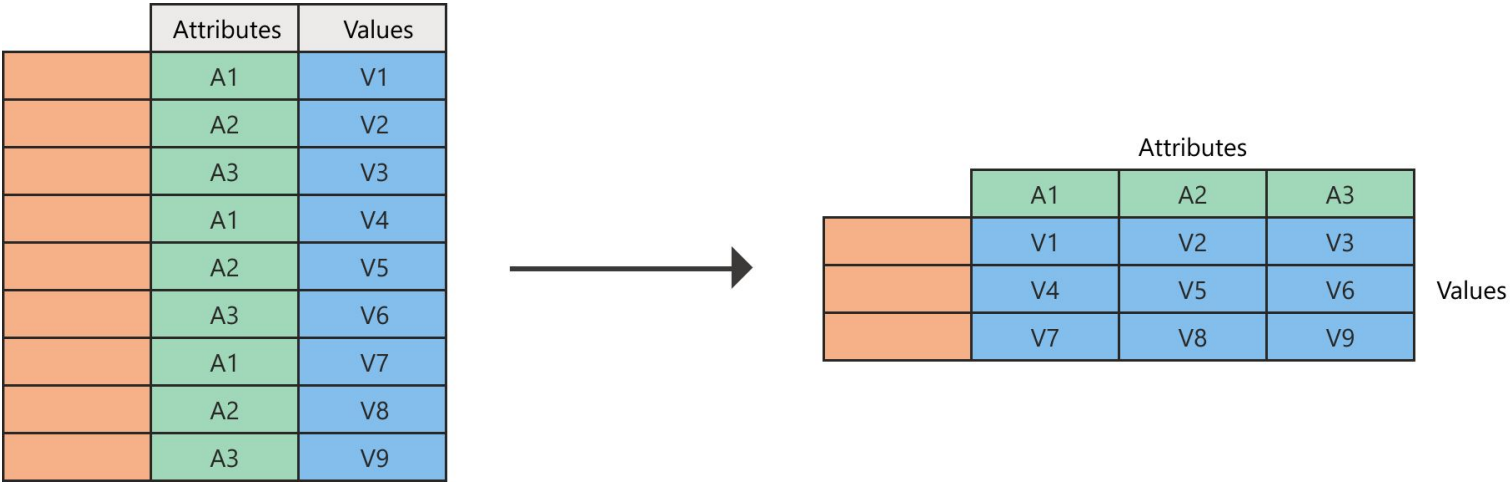
¿Por qué transformar la forma de los datos?

En la preparación de datos para Business Intelligence, no basta con tener la información correcta, también debe estar estructurada de manera que permita análisis eficientes y visualizaciones claras.

Existen dos formas predominantes de representar datos:

- Formato largo (long format): cada fila representa una observación única; es más flexible para aplicar filtros, agrupar, o generar medidas dinámicas.
- Formato ancho (wide format): cada columna representa una categoría o variable diferente; es más legible para reportes tabulares o dashboards visuales.

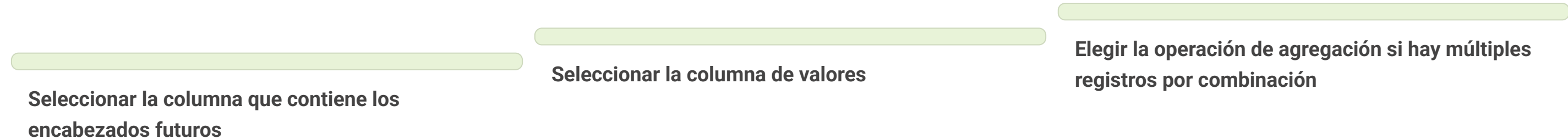
Contexto productivo: Una jefatura necesita ver, mes a mes, cuántas solicitudes ingresaron por comuna. Sin embargo, la tabla base tiene una fila por solicitud y una columna "Fecha". Para lograr esa visualización se necesita transformar la tabla desde formato largo a uno ancho, con un pivoteo por mes.



Pivoteo de columnas

Pivoteo significa convertir los valores únicos de una columna en encabezados de nuevas columnas. A cada nueva columna se le asigna un valor de acuerdo con otra columna relacionada (por ejemplo, conteo, suma o valor directo).

Subpasos clave



Seleccionar la columna que contiene los encabezados futuros

Seleccionar la columna de valores

Elegir la operación de agregación si hay múltiples registros por combinación

Ejemplo aplicado:

Deseamos generar una tabla con una fila por Comuna y una columna por Mes, con el total de solicitudes.

```
Table.Pivot(  
    TablaFiltrada,  
    List.Distinct(TablaFiltrada[Mes]),  
    "Mes",  
    "Total",  
    List.Sum  
)
```

Este código convierte la columna "Mes" en encabezados de columna y suma los valores de "Total" por comuna y mes.

Buenas prácticas en pivoteo

Preparación previa

Agrega una columna auxiliar (Mes, Año, Grupo) antes del pivoteo si los datos están en fecha continua.

Agregación explícita

Usa agregación explícita (List.Sum, List.Count) para evitar errores si hay múltiples valores por combinación.

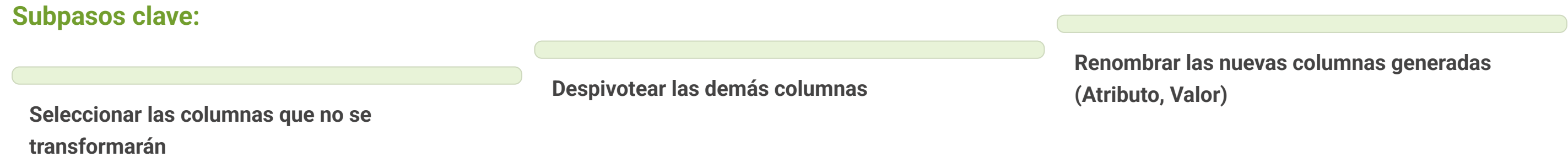
Validación de nombres

Valida que los nombres generados de columnas no se repitan o generen errores semánticos.

Despivoteo de columnas

Despivoteo (unpivot) convierte columnas en filas. Es útil cuando recibimos reportes en formato ancho pero necesitamos aplicar filtros o crear cálculos dinámicos sobre sus valores.

Subpasos clave:



Seleccionar las columnas que no se transformarán

Despivotear las demás columnas

Renombrar las nuevas columnas generadas (Atributo, Valor)

Ejemplo aplicado:

Queremos convertir las columnas Enero, Febrero, Marzo en una columna Mes y otra Total.

```
Table.UnpivotOtherColumns(  
    DatosMensuales,  
    {"Comuna"},  
    "Mes",  
    "Total"  
)
```

Este paso genera una estructura más simple que permite agrupar o segmentar por "Mes" sin necesidad de modificar el modelo.

Comparación entre pivoteo y despivoteo

Característica	Pivoteo	Despivoteo
Transforma...	Filas → columnas	Columnas → filas
Uso típico	Visualizaciones, dashboards	Limpieza, modelado, filtros
Requiere agregación	Sí, si hay duplicados	No necesariamente
Recomendado para...	Mostrar tendencias comparativas	Normalizar datos para análisis

Tabla 6. Comparación entre pivoteo y despivoteo

Fuente: Desafío Latam

Errores comunes y cómo evitarlos

Error	Consecuencia	Solución
Pivoteo sin agregación sobre valores duplicados	Error o resultados incorrectos	Aplicar List.Sum o filtrar duplicados antes
Despivoteo de columnas con distintos tipos de dato	Inconsistencias en la columna de valores	Convertir todas las columnas a tipo uniforme antes
Perder contexto al despivotear (por ejemplo, sin mantener la clave original)	Análisis incorrecto	Conservar columnas clave sin transformar

Tabla 7. Errores comunes y cómo evitarlos

Fuente: Desafío Latam

Dominar el pivoteo y despivoteo permite adaptar estructuras de datos a múltiples necesidades: visualización, análisis de tendencias, consolidación histórica o integración con calendarios. Es una herramienta transversal en Power Query, utilizada tanto en la preparación como en la optimización del modelo de datos.

Ejercicio - Consolidación y transformación de datos de atención ciudadana para análisis estratégico



Ejercicio

Instrucciones

Objetivo de la actividad

Aplicar en forma integrada las técnicas de anexado de datos, uniones (merge), agrupamiento y transformaciones estructurales (pivoteo/despivoteo) en Power Query para consolidar y estructurar un dataset operativo a partir de múltiples fuentes con formatos heterogéneos.

Se espera que las personas participantes comprendan cómo preparar información multifuente para visualización y análisis estratégico en un contexto real.



Ejercicio

Instrucciones

Situación problema

Trabajas en la Subdirección de Gestión Ciudadana de una red intercomunal. Tu equipo recibe cada mes los archivos con solicitudes ciudadanas desde distintas oficinas. Cada uno tiene ligeras variaciones: nombres de columnas diferentes, estructuras incompletas, o errores de capitalización en comunas.

Además, se cuenta con una tabla oficial de comunas válidas y una tabla de catálogo de servicios por tipo de solicitud.

Tu tarea es construir una tabla consolidada, depurada y lista para visualización en Power BI. Esta tabla debe mostrar el total de solicitudes por comuna, tipo de solicitud y mes, en formato ancho (una columna por mes).



Ejercicio

Instrucciones

1. Preparación inicial

Carga los archivos en Power BI.

Normaliza los nombres de columnas. Usa:

```
Table.TransformColumnNames(Enero,  
Text.Trim)
```

Agrega la columna "Mes" a cada archivo:

```
Table.AddColumn(Enero, "Mes", each  
"Enero")
```



Ejercicio

Instrucciones

2. Anexado de datos

Combina las tres tablas en una sola:

```
Table.Combine({Enero, Febrero, Marzo})
```



Ejercicio

Instrucciones

3. Merge con tablas de referencia

Une SolicitudesTotales con comunas_validas.csv:

```
Table.NestedJoin(  
  SolicitudesTotales,  
  {"Comuna"},  
  ComunasValidas,  
  {"Nombre"},  
  "ComunaValidada",  
  JoinKind.LeftOuter  
)
```

Expande columna:

```
Table.ExpandTableColumn(  
  Merge1,  
  "ComunaValidada",  
  {"Nombre"},  
  {"ComunaValidada"}  
)
```

Agrega columna de validación:

```
Table.AddColumn(  
  Merge1,  
  "Validez",  
  each if [ComunaValidada] = null then "Inválida"  
  else "Válida"  
)
```



Ejercicio

Instrucciones

4. Agrupamiento

Agrupa por Comuna, TipoSolicitud, Mes:

```
Table.Group(  
    Merge1,  
    {"Comuna", "TipoSolicitud", "Mes"},  
    {"Total", each Table.RowCount(_), Int64.Type}  
)
```


Ejercicio

Instrucciones

5. Pivoteo

Convierte los meses en columnas:

```
Table.Pivot(  
    Resumen,  
    List.Distinct(Resumen[Mes]),  
    "Mes",  
    "Total",  
    List.Sum  
)
```

6. Validación final

Verifica que las comunas inválidas estén marcadas.

Confirma que no haya registros duplicados por combinación Comuna + TipoSolicitud.

Asegúrate de que cada mes esté representado.

Resumen



Comprendimos la diferencia entre anexado (append) y merge (join) como formas de combinar datos.



Aplicamos operaciones de merge condicional para enriquecer una tabla con otra.



Agrupamos datos por atributos clave utilizando funciones de agregación.



Transformamos la estructura de las tablas mediante pivoteo y despivoteo.



Detectamos errores comunes al combinar y estructurar datos, y aplicamos buenas prácticas para evitarlos.

¿Qué buenas prácticas
aplicarías antes de realizar
un agrupamiento en Power
Query?





Próxima sesión...

En la siguiente sesión abordaremos el Modelado de datos. Exploraremos qué es un modelo de datos, cómo se estructuran las tablas de hechos y de dimensiones, y conoceremos los principales tipos de modelos como estrella y copo de nieve.